

Assignment D2-V1: Domestic Service Robot - Basic Coffee Preparation

Richard Albert King Mechoda
MSc Robotics Engineering, University of Genova
Artificial Intelligence for Robotics II

June 2026

Abstract

This report presents a planning model for a fixed robotic manipulator that prepares a cup of coffee in a kitchen. Question 1 models the task in classical PDDL (STRIPS with typing): object states are represented as a chain of predicates and the correct recipe order emerges purely from action preconditions. Two problem instances are provided, and both were solved with the `pyperplan` planner (plans of length 3 and 9). Question 2 extends the domain to PDDL+: a continuous process models the coffee cooling over time and an event makes the coffee unusable once it reaches a temperature threshold, so the validity of a plan now depends on timing. Both PDDL+ instances were verified with ENHSP: with a fast delivery the problem is solved, with a slow delivery the planner proves it unsolvable, although the two instances differ in a single duration value. The report also documents a modelling loophole that ENHSP exploited in an early version of the domain, and discusses the limits of discrete models for physical processes.

1 Problem formulation

A single fixed manipulator operates in a kitchen containing a cup, a water source, a coffee powder dispenser and a coffee machine. The robot must obtain the empty cup, fill it with water, add coffee powder and use the machine to prepare the coffee. All objects start at fixed positions and the robot can manipulate one object at a time.

The assignment asks for (Q1) a classical PDDL model with a simple and a more complex problem instance, both with valid plans and justified modelling choices, and (Q2) a PDDL+ extension with a process for the coffee cooling over time, an event for the coffee becoming cold and unusable, and a demonstration of how timing affects plan validity.

2 Q1 - Classical PDDL model

2.1 Design choices

Only the cup is manipulated. The water tap, the powder dispenser and the machine never move, so instead of modelling them as objects they are modelled as *capabilities of locations*: `(is-water-source ?l)`, `(is-powder-source ?l)`, `(is-machine ?l)`. This keeps the model within the single-object manipulation constraint and avoids modelling water as a graspable thing, which would drag in containers, pouring and spilling. The same kind of abstraction is used in the assembly example from the lectures, where the screwdriver is abstracted away: the model captures causality, not physics.

Object states form an explicit chain. The contents of the cup are represented by four predicates,

```
(empty ?c) -> (has-water ?c) -> (has-powder ?c) -> (coffee-ready ?c)
```

each of which records that one transformation has happened and gates the next one. This satisfies the guideline to clearly distinguish object states (empty / filled / prepared) and avoids collapsing several steps into one action.

Grasping and placing are explicit. `grasp` and `place` are separate actions operating on a one-slot gripper, represented by `(gripper-empty)` and `(holding ?c)`. Every station operation additionally requires the cup to be *placed* at the right location with the gripper *free* (the arm needs its hand to operate the tap or press the machine button). This makes `grasp` and `place` genuinely necessary steps rather than decoration.

Ordering is enforced by preconditions only. No ordering is written explicitly anywhere; it emerges from the preconditions, as summarised in Table 1. One choice worth flagging: water before powder is physically arbitrary, but the assignment text gives that exact order, so the precondition of `add-powder` enforces it.

Action	Key precondition	Effect on ordering
<code>fill-water</code>	<code>(empty ?c)</code>	only an empty cup can be filled
<code>add-powder</code>	<code>(has-water ?c)</code>	powder only after water
<code>brew-coffee</code>	<code>(has-water ?c), (has-powder ?c)</code>	machine needs both

Table 1: How preconditions induce the recipe order.

2.2 Problem instances and plans

Both instances use the same domain; only the initial state differs.

Simple instance (`q1_problem_simple.pddl`). A single counter provides all three capabilities and the cup already sits on it, so no transport is needed. `pyperplan` finds the minimal 3-step plan:

```
(fill-water cup1 counter)
(add-powder cup1 counter)
(brew-coffee cup1 counter)
```

Complex instance (`q1_problem_complex.pddl`). The three stations are distinct locations and the cup starts on a separate shelf. Since every operation needs the cup placed at its own station, the robot must move the cup repeatedly. `pyperplan` finds a 9-step plan:

```
(grasp cup1 shelf)
(place cup1 water-source)
(fill-water cup1 water-source)
(grasp cup1 water-source)
(place cup1 powder-dispenser)
(add-powder cup1 powder-dispenser)
(grasp cup1 powder-dispenser)
(place cup1 coffee-machine)
(brew-coffee cup1 coffee-machine)
```

The contrast between the two instances illustrates a point from the lectures: the difficulty of a planning problem lives in the structure of the instance, not in the actions, which are identical in both cases.

3 Q2 - PDDL+ model

3.1 What changes with respect to Q1

Classical PDDL assumes the world is frozen between actions, so waiting is free. Real coffee cools down regardless of what the robot does. The PDDL+ model therefore adds a numeric fluent (`temperature ?c`) together with:

A cooling process. While hot coffee exists, its temperature decreases at a constant rate (a deliberate linear simplification of Newton’s law of cooling):

```
(:process cooling
  :parameters (?c - cup)
  :precondition (coffee-ready ?c)
  :effect (decrease (temperature ?c) (* #t 1)))
```

The process runs autonomously; the planner never triggers it. Doing nothing now has consequences.

A cold event. The instant the temperature reaches the threshold (`drinkable-temp`), the coffee becomes cold and unusable:

```
(:event coffee-got-cold
  :parameters (?c - cup)
  :precondition (and (coffee-ready ?c)
                    (<= (temperature ?c) (drinkable-temp)))
  :effect (and (not (coffee-ready ?c)) (spoiled ?c)))
```

Events fire automatically at threshold crossings, with no planner involvement, which is exactly the hybrid-system behaviour discussed in the lectures.

3.2 Timed activities without durative actions

ENHSP, the planner used for validation, supports processes and events but not durative actions (the planners that support durative actions, such as OPTIC or POPF, conversely do not support processes). Brewing and delivery, which both take time, are therefore encoded with the standard PDDL+ pattern: an instantaneous *start* action sets a flag and resets a clock, a *process* advances the clock while the flag holds, and a completion *event* fires once the clock reaches the duration. Semantically this behaves like a durative action. Brewing takes 5 time units and ends with hot coffee at 90 °C; delivery takes (`delivery-time`) units and succeeds only if (`coffee-ready`) still holds when the hand-over completes.

3.3 A loophole found by the planner

The first version of this domain produced a surprise: ENHSP “solved” the slow-delivery instance that was meant to be unsolvable. Its plan started the delivery, let the coffee go cold in transit, then placed the cup back into the machine *during* the delivery and re-brewed the spoiled coffee; since the delivery clock had already expired, the completion event fired the moment the coffee was hot again. Physically this is nonsense, but nothing in the model forbade it. Two preconditions close

the loophole: `place` now requires `(not (delivering ?c))` (a started hand-over is a committed motion), and `start-brew` requires `(not (spoiled ?c))` (spoiling is irreversible). The episode is a concrete instance of the warning from the lectures that a model must encode all of its invariants, because nothing is enforced automatically: a planner behaves like an adversarial optimiser and will exploit any physical constraint left implicit.

3.4 How timing affects plan validity

The two PDDL+ instances are identical except for the value of `(delivery-time)`. Coffee is brewed at 90°C, cools at 1°C per time unit, and the user accepts at least 40°C, leaving a 50-unit hot window after brewing.

Fast delivery (`delivery-time = 10`). ENHSP reports **Problem Solved** with the time-stamped plan

```
0.0: (fill-water cup1 counter)
0.0: (add-powder cup1 counter)
0.0: (start-brew cup1 counter)
      [t = 5: event brew-done, temperature := 90]
5.0: (grasp cup1 counter)
5.0: (start-delivery cup1)
      [t = 15: event delivery-complete, temperature ~ 80]
```

The coffee arrives at roughly 80°C and the goal (`served cup1`) is reached.

Slow delivery (`delivery-time = 70`). The delivery needs 70 units but the hot window is only 50: whenever the delivery starts, the cold event fires in transit, retracts (`coffee-ready`), and the completion event can never trigger. ENHSP exhausts the search space and reports **Problem unsolvable**. A single changed duration flips the instance from solvable to unsolvable, which is precisely how timing affects plan validity. The full planner outputs for both instances are included in the repository.

4 Discussion

4.1 Abstraction of object states

In Q1 the entire physical content of the cup is compressed into four boolean predicates. The benefit is enormous: the task becomes a small, decidable search problem and the whole recipe order is encoded in preconditions without any explicit script. The cost is that the abstraction cannot express quantities: how much water, how warm, how well mixed - a cup either **has-water** or it does not. These predicates are causal milestones, not physical measurements. Moving from Q1 to Q2 is exactly the act of replacing one such boolean (hot versus cold) with a numeric quantity when the boolean version is too coarse. The loophole of Section 3.3 shows the danger from the other side: every fact the abstraction leaves out (that spoiling is irreversible, that a cup being delivered is physically in the robot's hand) is a fact the planner is free to violate.

4.2 Limits of discrete modelling for physical processes

Classical PDDL changes state only when the planner acts, so waiting is free and harmless; a cooling cup breaks that assumption. Discrete workarounds (for instance a `let-cool` action) are wrong twice: they let the planner decide whether physics happens, and they cannot capture the instant a threshold is crossed. PDDL+ repairs this with processes and events, turning planning into reasoning over trajectories rather than sequences of snapshots. The limits remain visible

even in this small model: ENHSP discretises the dynamics (with $\Delta t = 1$ by default), so plans are valid only with respect to that discretisation; the constant cooling rate is a linear stand-in for an exponential law; durative actions had to be compiled by hand into start-process-event triples because no common planner accepts both dialects; and the model still hides all the geometry and grasp feasibility of real manipulation, with no sensing or uncertainty about the true temperature. Every layer of added expressiveness costs search efficiency, so choosing the right abstraction for the question being asked is itself the central modelling decision.

5 Reproducibility

All models are in the `codes/` folder and the planner outputs are committed under `codes/outputs/`. Q1 was validated with `pyperplan` (`pip install pyperplan`):

```
pyperplan q1_domain.pddl q1_problem_simple.pddl
pyperplan q1_domain.pddl q1_problem_complex.pddl
```

The closing `Plan correct` line is produced by VAL, an independent plan validator that `pyperplan` invokes automatically. Q2 was validated with ENHSP-20, built from the `enhsp20-0.9.0` tag of the public repository with JDK 17:

```
java -jar enhsp.jar -o q2_domain.pddl -f q2_problem_intime.pddl
java -jar enhsp.jar -o q2_domain.pddl -f q2_problem_late.pddl
```

Two practical notes for anyone repeating this: the predicate name `at` is reserved in the ENHSP parser (the Q2 domain uses `cup-at`), and the requirements list must stay minimal, since tokens such as `:time` or `:negative-preconditions` are rejected by its grammar even though negative preconditions themselves work.